

Denial Dashboard & Denial Workflow

Requirements and technical specification for the two denial modules and the data pipeline that feeds them. Written against the code as it stands after the August 2026 export and Report Control Board changes.

Revision 2.0 **Date** 11 August 2026 **Status** Current – reflects deployed behaviour
Applies to LabMetricsDashboard · LRN.ReportsApi · LRN.ReportWorker · LRN.DenialDatabaseWorker

CONTENTS

- 1 Purpose & scope
- 2 System context
- 3 Shared data model
- 4 Denial Database Worker
- 5 Denial Dashboard
- 6 Denial Workflow
- 7 Task lifecycle
- 8 Roles & permissions
- 9 Exports
- 10 Non-functional
- 11 Open items & risks
- 12 Change log

1 Purpose & scope

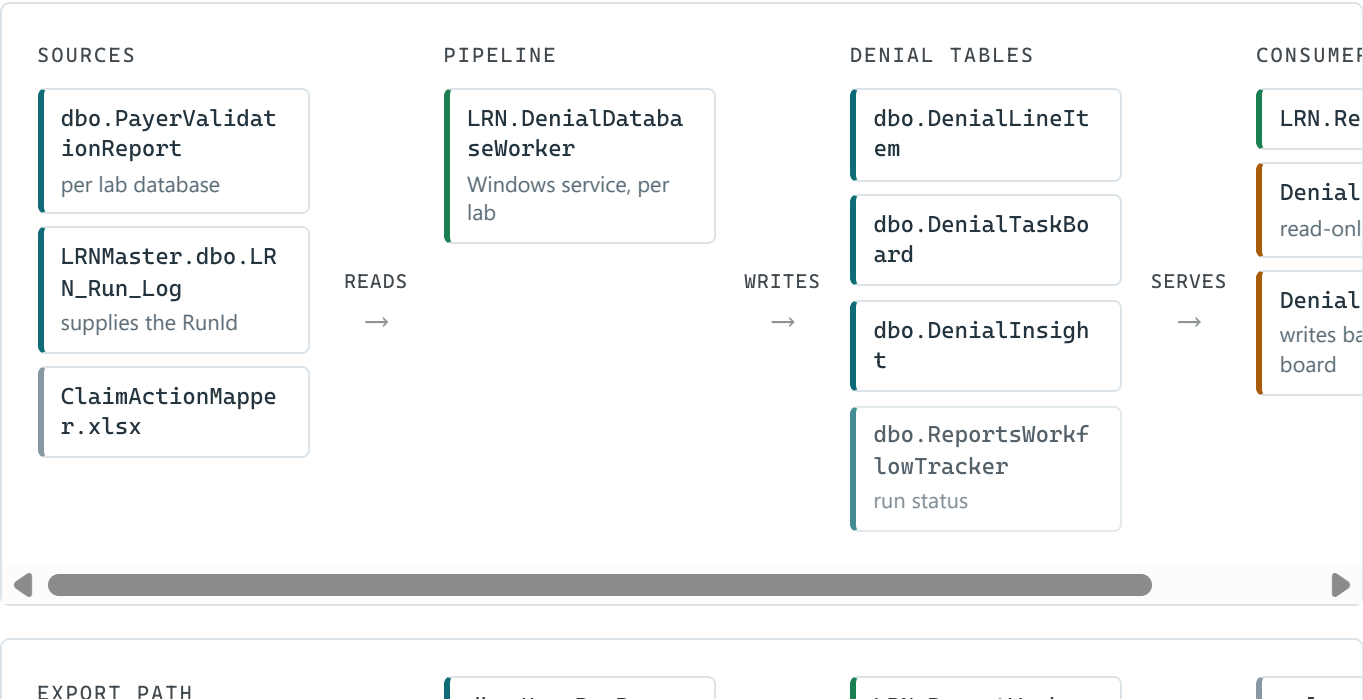
Denial management in Lab Revenue Navigator spans three deployables and two user-facing modules. This document specifies all of them together, because neither module can be understood — or tested — without the pipeline that populates it.

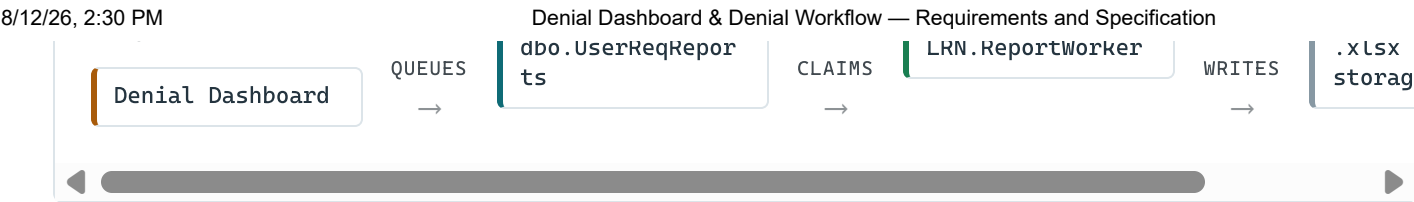
MODULE	WHO USES IT	WHAT IT IS FOR
Denial Dashboard /DenialDashboard	RCM leads, AR managers, analysts	Read-only analysis of one lab's denials for the latest run: trends, SLA exposure, denial-code insight, and the full line-item grid. The reporting surface.
Denial Workflow /DenialWorkflow	AR managers, AR reviewers/analysts	Operational queue. Assign denials to reviewers, work them to a status, and clear verification items. The system of record for who did what.
Denial Database Worker LRN.DenialDatabaseWorker	Unattended service	Turns each lab's <code>PayerValidationReport</code> rows into the three denial tables both modules read.

Out of scope

- Production of `dbo.PayerValidationReport` itself — an upstream process owns it.
- The Payer Policy validation engine and coding validation, which supply columns consumed here but are specified separately.
- Report Control Board status semantics, except where the denial pipeline reports into it (§4.6).

2 System context





COMPONENT	RUNS AS	REACHES THE DATA BY
LabMetricsDashboard	IIS web app	HTTPS to LRN.ReportsApi, JWT minted from the signed-in user
LRN.ReportsApi	IIS web app	Direct SQL. Resolves each lab from <code>LRNMaster.dbo.LRNMetricsLab</code> → <code>ConnectionKey</code>
LRN.ReportWorker	Windows service	Direct SQL to the lab database supplied by the queued job's lab config
LRN.DenialDatabaseWorker	Windows service	Direct SQL: LRNMaster for the run, then the lab database

WHY THE WORKER DOES NOT USE THE API

The API client mints its JWT from the signed-in `HttpContext`. A background service has no such context, so `LRN.ReportWorker` reads the denial tables over direct SQL instead. Both paths must return the same rows; §11.2 covers the case where they do not.

3 Shared data model

All denial tables live in the **lab's own database**, one set per lab, scoped by `LabId` and `RunId`.

TABLE	GRAIN	WRITTEN BY	READ BY
PayerValidationReport	One row per claim line, every pay status	Upstream process	Denial Database Worker
DenialLineItem	One row per denied claim line	Worker (bulk)	Dashboard — Line Item, breakdowns
DenialTaskBoard	One row per actionable denial task	Worker (bulk); Workflow (updates)	Dashboard — SLA Tracker, Filter Panel; Workflow — Task Board

DenialInsight	One row per denial code × payer	Worker (bulk)	Dashboard and Workflow — Denial Insight
DenialVerificationTask	One row per task awaiting verification	Workflow	Workflow — Verification
DenialTaskHistory	One row per action, with a JSON snapshot	Workflow	Audit
DenialAnalysisRunLog	One row per completed pipeline run	Worker	Worker (idempotency guard)
DenialActionCategoryMaster	Reference — action category → scope	Maintained	Workflow (claim-level propagation, §7.4)

3.1 Key linkage

`DenialLineItem` and `DenialTaskBoard` share no surrogate key. The task board stores the claim as `ClaimID`; the line item stores `VisitNumber`; one side may carry a `CLM-` prefix. Any join between them must normalise that prefix and match on claim + CPT + denial code.

`UniqueTrackId` is `ClaimUID | CPTCode | DenialCode` and is the pipeline's identity for a task across runs.

4 Denial Database Worker

4.1 Requirements

DW-01	Each lab is processed independently; one lab's failure must not stop the others.
DW-02	The service must never invent a RunId. It attaches to the lab's most recent successful run from <code>LRNMaster.dbo.sp_GetRecentSuccessRunByLab</code> .
DW-03	Only rows whose <code>PayStatus</code> equals the configured denied value (default <code>Denied</code>) are imported.

DW-04 A run that already succeeded must not be reprocessed. A previous **Failed** or **Skipped** run must be retried.

DW-05 Every run reports **InProgress** → **Success** / **Failed** / **Skipped** to `dbo.ReportsWorkflowTracker` under the report name Denial Report.

DW-06 Reaching the end with nothing to import is a **skip**, not a failure, and must state why in terms an operator can act on.

DW-07 The Excel/CSV/ZIP export package and its SharePoint upload are optional and default to off; the service's primary job is the SQL copy.

4.2 Run selection and the RunId contract

The RunId comes from LRNMaster; the rows come from the lab table. Those two must agree, and this is the single most common cause of a run producing nothing.

1. Take the latest successful RunId for the lab from LRNMaster.
2. Seek `WHERE RunId = @RunId` in `PayerValidationReport` — a bare comparison, so an index on `RunId` is a seek.
3. On a miss, resolve the table's own spelling with `LTRIM(RTRIM(RunId)) = @RunId` and re-aggregate on that exact literal.
4. All later data queries use the **stored** spelling. Tracking and logging stay on LRNMaster's value.

WHY STEP 3 EXISTS

SQL Server ignores *trailing* blanks in `=` but not *leading* ones. A RunId stored as `' R2026...'` silently matches nothing, so `SELECT ... WHERE PayStatus = 'Denied'` returns thousands of rows while the service reports none. If the two systems hold genuinely different ids, no trimming helps — the skip message now lists the RunIds the table actually holds so the mismatch is visible.

4.3 Pipeline steps

Each step is written to the CSV step log and to `dbo.ReportRunIdInfoLog`.

1 Start

Open the info log, mark the tracker InProgress.

- 2

Load ClaimActionMapper
Latest mapper workbook for the lab. Absence is a hard failure.
- 3

Load denied rows
Read `PayerValidationReport` for the resolved RunId, keyed back to the original Excel header names.
- 4

Normalize and map
Normalise the denial code, apply the action mapper.
- 5

Build insight rows
Aggregate to denial code × payer.
- 6

Build task board rows
Derive tasks, priority, SLA and due dates.
- 7

Write export package
Optional. Skipped when `GenerateOutputFiles` is false.
- 8

Write tables
Bulk write insight, line item and task board; carry forward assignments and raise verification items for tasks that vanished from the new run.
- 9

Upload to SharePoint
Optional.
- 10

Completed
Record the run in `DenialAnalysisRunLog`, mark the tracker Success, notify Teams.

4.4 Skip conditions

CONDITION	OUTCOME	OPERATOR READS
No successful run in LRNMaster	<code>exit quietly</code>	Nothing to process — no tracker row, because both log procedures reject a blank RunId.
Report already succeeded for this RunId	<code>Skipped</code>	Already succeeded. Skipping.
RunId has no rows in the lab table	<code>Skipped</code>	No rows for that RunId, plus the RunIds the table does hold.
RunId already in	<code>Skipped</code>	Already recorded.

DenialAnalysisRunLog		
Rows exist but none are denied	Skipped	Total row count found, none with the denied pay status.

4.5 Configuration

SETTING	DEFAULT	EFFECT
PayerValidationReportTable	dbo.PayerValidationReport	Source table. Validated as schema.table .
DeniedPayStatus	Denied	The only pay status imported.
ProcessLatestRunOnly	true	true filters every read by RunId. false imports every denied row, stamped with the newest RunId.
GenerateOutputFiles	false	Excel/CSV/ZIP package. Off = pure SQL copy.
ReportName	Denial Report	Must match dbo.ReportTypeMaster exactly or the tracker upsert is rejected.
SqlCommandTimeoutSeconds	600	The 30 s ADO.NET default is far too short for a whole-run read on a large lab.

4.6 Reporting into the Report Control Board

The tracker row this worker writes is what the board renders in its **Denial** column. A skip leaves the column without a success, which the board now shows as `in progress` and, more than a day after the run's source data landed, `overdue` .

5 Denial Dashboard

5.1 Requirements

DD-01	Show one lab at a time, always for that lab's current run.
DD-02	Every tab honours the same global filter set; a filter change must not silently apply to some tabs and not others.

DD-03	An AR Reviewer sees only denials assigned to them, and only the Task Board and Line Item tabs.
DD-04	The download must be produced in the background and must not block or time out the request.
DD-05	Line Item and Denial Insight grids page server-side and support per-column filters across the whole dataset, not just the loaded page.

5.2 Tabs

TAB	SOURCE	SHOWS
Dashboard	Task board	Summary tiles — open, overdue, high priority, balance.
Weekly Breakdown	Line item	Pivot by week of <code>DenialDate</code> : top insurance and top denials.
Monthly Breakdown	Line item	Same pivot, monthly, with year groupings.
Filter Panel	Task board	Breakdown panels: status, priority, action category, classification, deadline.
Task Board	Task board	Full task grid with reviewer assignment.
SLA Tracker	Task board	Deadline monitor with days remaining and escalation flag.
Denial Insight	Insight	Denial code × payer with balance impact; managers can assign reviewers.
Line Item	Line item	Excel-style grid over every <code>DenialLineItem</code> column.

Denial Input is retired; old links redirect to Dashboard.

5.3 Filters

Multi-choice filters travel pipe-delimited with `(ALL)` as the no-filter sentinel. Dates are inclusive.

- **Choice:** Status, Priority, Action Category, Deadline, Classification, Denial Code.
- **Text:** Sales Rep, Clinic, Referring Provider, Payer Name, Payer Type, Panel Name.
- **Date ranges:** First Billed, Date of Service, Denial Date.
- **Per-column grid filters** applied server-side on top of the above.

6 Denial Workflow

6.1 Requirements

WF-01	Only a manager or admin may assign work; only a reviewer or admin may change a task's status.
WF-02	Every assignment, status change and verification decision writes a history row with a snapshot of the task before the change.
WF-03	Assignment is by denial code × payer , not by hand-picking rows — a reviewer owns a denial pattern for the run.
WF-04	Marking a task Duplicate removes it from the task board and raises a verification item; it must not be silently closed.
WF-05	All writes for one action are a single transaction; a partial assignment must never be observable.
WF-06	A reviewer's own queue is the default view; managers see all reviewers with a per-reviewer summary.

6.2 Tabs

TAB	VISIBLE TO	PURPOSE
Dashboard	All	Counts: assigned, pending, completed, verification pending, unassigned — plus a per-reviewer summary.
Denial Insight	Admin, AR Manager	Assign reviewers to denial code × payer rows, singly or in bulk.

Task Board	All (reviewers see only their own)	Work a task: set status, add comments.
Verification	All	Decide whether a removed or duplicate denial was valid.

6.3 Filters

Status, reviewer / assigned-to, denial code, payer name. Reviewer and assigned-to are the same filter under two names; whichever is supplied wins. Page size is 100 on screen, 50,000 for export.

7 Task lifecycle

7.1 Statuses

STATUS	MEANING	SET BY
New	Created by the pipeline, unassigned.	Worker
Pending Review	Assigned, not yet started.	Assignment (automatic)
In-Progress	Reviewer is working it.	Reviewer
Verification Pending	Awaiting a verification decision.	Duplicate action, or the pipeline
Duplicate	Transitional — moves the task to verification.	Reviewer
Completed	Worked and finished.	Reviewer
Closed	Finished, no further action.	Reviewer

7.2 Transitions

ACTION	PRECONDITION	EFFECT
Assign	Manager or admin	Sets AssignedTo . Status moves to Pending Review only if it is blank or New — an in-flight status is never overwritten. History: Assigned .
Update	Reviewer or admin	Sets status and reviewer comments. Closed/Completed stamps

status		DateCompleted . History: StatusChanged .
Mark duplicate	Reviewer or admin	Copies the task into DenialVerificationTask and deletes it from the task board . History: StatusChanged .
Verify — valid	Manager, reviewer or admin	Re-inserts the task into the task board under the chosen status. History: VerifiedValid .
Verify — invalid	Manager, reviewer or admin	Task is not restored; history only. History: VerifiedInvalid .

Either verification decision closes the verification row (stamping status, comments, verifier and time) and then deletes it.

7.3 Verification raised by the pipeline

When a new run no longer contains an **assigned, open** task from the previous run, the pipeline flags it ClaimFrom = 'Old Run' , WorkFlowStatus = 'Verification Pending' and raises a verification row. This is how a denial that silently disappears between runs still gets a human decision. See §11.1 for a naming discrepancy affecting this path.

7.4 Claim-level propagation

If the task's action category is registered in DenialActionCategoryMaster with ActionScope = 'ClaimLevel' , the status change applies to **every task on the same ClaimID** , not just the one row. Line-level categories affect only that row.

8 Roles & permissions

Role names are matched with and without spaces, and Analyser/Analyzer spellings are treated as equivalent, because the identity source is inconsistent.

CAPABILITY	ADMIN	AR MANAGER	AR REVIEWER / ANALYST
View Denial Dashboard — all tabs	Yes	Yes	Task Board and Line Item only
See other reviewers' denials	Yes	Yes	No – own only
Assign reviewers	Yes	Yes	No
Change task status	Yes	No	Yes

Decide verification	Yes	Yes	Yes
Workflow Denial Insight tab	Yes	Yes	No
Export	Yes	Yes	Yes

NOTE

An AR Manager can assign work but cannot change a task's status; a reviewer is the opposite. That separation is deliberate and enforced server-side in the controller, not only in the view.

9 Exports

9.1 Denial Dashboard workbook

Queued to `dbo.UserReqReports` and generated by `LRN.ReportWorker`. The  Reports badge tracks progress and serves the file. If a lab has no queue deployed, the button falls back to a synchronous download.

Exactly six sheets, in tab order. Every sheet is written even when empty, so the shape never changes between runs.

#	SHEET	CONTENTS
1	Monthly Breakdown	Monthly pivot, unchanged formatting.
2	Weekly Breakdown	Weekly pivot, unchanged formatting.
3	Filter Panel	Run id, filtered task count, active filters, then breakdowns: Task Status, Priority, Assigned To , Action Category, Classification, Deadline.
4	SLA Tracker	Deadline monitor in page column order.
5	Denial Insight	Pre-aggregated insight rows.
6	Line Item	Every line-item column, plus the four workflow columns below.

WORKFLOW COLUMNS ON LINE ITEM

WORKFLOW COLUMNS ON LINE ITEM

`Workflow Task ID`, `Assigned To`, `Workflow Status`, `Denial Notes` (the reviewer's comments). Matched from the task board on normalised claim + CPT + denial code, falling back to claim + CPT. A claim-only fallback is deliberately **not** used — one claim spans many CPTs that can belong to different reviewers, so it would attach the wrong reviewer's notes. Unmatched lines are blank, never wrong.

OVERSIZED CELLS

Excel rejects any cell over 32,767 characters. Values past that — NorthWest's `Covered ICD 10 codes as per Payer Policy` is the known case — spill into `... (cont. 1)`, `(cont. 2)` ... columns, split at a comma so a code is never cut in half. Capped at five spill columns; beyond that the final cell carries a visible truncation marker. Labs with no oversized values get exactly the sheet they always got.

FILE NAME

`Denial_<Lab>_<RunId>_<WeekFolder>.xlsx`

Missing parts collapse out rather than leaving empty separators.

9.2 Denial Workflow workbook

Synchronous, one sheet for the active tab (Insight, Task Board or Verification), named `DenialWorkflow_<Lab>_<tab>_<timestamp>.xlsx`. Export page size is 50,000 rows.

9.3 Formatting rules

- Green header (`#548235`), white bold text, frozen header row, AutoFilter.
- **No zebra banding** on any data sheet. Weekly and Monthly Breakdown keep their existing pivot formatting.
- Money as `$#,##0.00` ; dates as `yyyy-mm-dd` .

10 Non-functional requirements

AREA	REQUIREMENT	HOW IT IS MET
Memory	A single lab's export must not be bounded by available RAM.	The Line Item sheet is streamed row-by-row via OpenXml. ClosedXML buffers a whole sheet in a <code>MemoryStream</code> and fails past 2 GB.
Sheet size	No worksheet must exceed 1,048,576 rows.	Sheets split at 200,000 rows max. 200 MB of estimated XML

Sheet size	no worksheet part may approach 2 GB.	Sheets split at 200,000 rows or ~500 MB of estimated XML, whichever comes first. A row cap alone is not enough when rows carry 72 KB ICD lists.
Query time	Batch reads must not fail on the largest labs.	600 s command timeout in the worker; RunId predicates kept sargable.
Security	Every action authorised server-side; reports never enumerable.	Global authorize filter; role checks in each POST action; queued reports scoped to the requesting user and guarded by an unguessable download token; files stored outside the web root and streamed.
Auditability	Every workflow action attributable.	<code>DenialTaskHistory</code> row per action with a before-snapshot, actor and run id.
Degradation	A missing table or an unreachable lab must not blank the page.	Table-existence guards return empty sets; the tracker keeps a last-good copy behind an error card.

KNOWN TRADE-OFF

The "table missing → empty list" guards mean a **wrong connection** looks exactly like an **empty table**. The export now probes for `DenialLineItem` up front and fails loudly, and logs a census of all three tables when the task board comes back empty.

11 Open items & risks

11.1 Verification table naming

NEEDS CONFIRMATION

LRN.DenialDatabaseWorker writes pipeline-raised verification rows to `dbo.DenialVerification`. Both LRN.ReportsApi and the dashboard's own repository read and write `dbo.DenialVerificationTask`. Unless a synchronisation step exists that this review did not find, verifications raised by the pipeline (§7.3) never appear in the Verification tab — meaning denials that vanish between runs are silently dropped rather than decided.

11.2 Task board empty for some labs

UNDER INVESTIGATION

For some labs — NorthWest reported — a queued export produces a populated Line Item sheet but empty SLA Tracker, Filter Panel and Denial Insight sheets. All three derive from `DenialTaskBoard` (Denial Insight falls back to task-board-derived rows when `DenialInsight` is empty), so one empty table explains all three.

The worker reaches the tables through the lab's `DbConnectionString` from its config JSON, while the web path resolves them through LRNMaster's `ConnectionKey`. The two candidates are that the tables live in a different database, or under a different `LabId`. A census is now logged on the empty path naming the database, table existence and per-lab row counts; one failed run will settle it.

11.3 Unmapped labs

CONFIGURATION GAP

The Report Control Board currently shows every lab as **unmanned** — the tracker's lab